



Capstone Project AI-Enhanced React Application Task

[Visit our website](#)

Introduction

Use your React knowledge to create a polished web application for your developer portfolio. This task will guide you through designing a functional and visually appealing app while applying core concepts in React development. You will also demonstrate your ability to integrate AI-powered functionality into a React application while following responsible AI design principles.

Throughout this project, you will apply concepts covered in React development, state management, AI API integration, AI-driven interfaces, user experience design, and conversational and adaptive user interfaces. You will also set up a Git repository to manage version control throughout the project.



Practical task

For this capstone project, you will build a web application that helps users organize and manage personal or professional events. This task will consolidate your knowledge of React, JSX, JavaScript, React Hooks, routing, form validation, state management, and AI integration while showcasing your ability to design a functional and engaging application.

The Personal Event Planner will allow users to schedule, view, and track events such as appointments, meetings, workshops, conferences, or social gatherings. The application should meet the following criteria.

1. Create user registration and login features

Users should be able to create an account by providing their name, email, username, and password. Input validation should ensure that:

- Fields are not left empty.
- Email addresses follow the correct format.
- Passwords meet any validation requirements you choose to implement.

Once registered, users should be able to log in and access their dashboard. The dashboard should display their upcoming events in an organized view.

2. Build event management functionality

Users should be able to add new events by providing details such as:

- Event name
- Date
- Time
- Description
- Location

Events should appear on the dashboard in an organized view, such as a list or calendar format. Use React's `array.map()` method to dynamically generate event displays based on the data stored in the application. Users should be able to update event details or remove events entirely. Any changes made should immediately update the application's state so that displayed information always reflects the current data.

3. Add an AI-powered event assistant

To demonstrate your understanding of AI integration in modern web applications, your event planner must include at least one AI-powered feature that helps users create, improve, or manage events more effectively. Examples of acceptable AI features include:

- Generating an event description from a short event title.
- Creating a meeting agenda based on event details.
- Suggesting preparation tasks for an event.
- Generating reminders or follow-up actions.
- Recommending event categories or tags.
- Providing event organization suggestions.

For example, a user might enter an event title and select a "Generate Description" button. The application then sends the request to an AI service and displays the generated response inside the event form. You may use a free AI service such as [OpenRouter](#) or the [Google Gemini API](#). OpenRouter is recommended because it provides a simple API interface that can be accessed using standard JavaScript `fetch()` requests.



Take note

If you are unable to access an AI API, you may implement a simulated AI feature that demonstrates asynchronous AI interactions, generated responses, loading states, and error handling. For example, you could simulate AI-generated event descriptions or recommendations using JavaScript and delayed responses..

Regardless of the implementation approach, the AI feature must demonstrate:

- Asynchronous request handling.
- User feedback while content is being generated.
- Error handling if generation fails.
- Clear communication that content is AI-generated.
- User control over generated content before saving.

AI user experience requirements

Your AI feature must include:

- A loading state while waiting for the AI response.
- Error handling if the AI service fails or becomes unavailable.
- Clear labeling indicating that the content was generated by AI.
- A way for users to edit AI-generated content before saving it.

Responsible AI requirements

Because AI systems can generate inaccurate or incomplete information, your application should include a short notice informing users that AI-generated content should be reviewed before use. An example message could be:

"This content was generated by AI and may contain inaccuracies. Please review before saving."

4. Create application navigation

Include a fixed header at the top of the application that remains visible during navigation. The navigation menu should contain links to important sections such as:

- Dashboard
- Add Event
- Help

You may include additional navigation options if they improve the user experience.

5. Manage application state

Use the Context API to manage:

- User authentication information.
- Event data.
- Any shared application state required by your AI-powered feature.

The Context API should be used wherever appropriate to avoid unnecessary prop drilling.

6. Create user help center

Add a Help section that provides clear guidance for users. The Help page should explain:

- How to register an account.
- How to log in.
- How to create events.
- How to edit events.
- How to delete events.
- How to use the AI-powered feature.
- How AI-generated content should be reviewed before use.
- Tips for organizing events effectively.

7. Design a responsive user interface

Design the application so that it adapts smoothly to different screen sizes, including desktop, tablet, and mobile devices. The interface should remain user friendly and visually appealing across devices. You may use:

- Custom CSS
- CSS Modules
- React-Bootstrap
- Another suitable styling approach

The final application should demonstrate professional front-end design practices.

8. Track development using Git

Track project development using Git. Create a Git repository and commit changes regularly throughout the project. Include a **README.md** file explaining:

- The purpose of the application.
- How to install the project.
- How to run the project.
- The AI feature you implemented.
- Any AI service or API used.
- Known limitations of the AI functionality.

Keep the documentation clear and concise.

9. Build the application using React and Vite

Your application must be built using React and Vite. The reviewer should be able to launch your project by navigating to the project directory and running:

```
npm install
npm run dev
```

The application should start successfully without requiring modifications to the source code. Your submission must include:

- All React source code files.
- Package configuration files.
- Any styling files used in the project.
- README.md documentation.
- Any assets required for the application to function correctly.

10. Organise and refine your codebase

The file structure of the project should be well organised and in line with [these guidelines](#). The code should also be easy to read, adhering to [Google's style guide](#) about indentation, meaningful variables, component names, etc.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
